

LLM Agent 年度研究总论

从模型能力竞赛转向系统工程、长时状态、可验证执行与现实工作流

研究范围	2025-08-20 至 2026-08-19; 另列少量边界基线
语料规模	70篇论文/预印本, 45篇官方或作者工程博客, 36个开源仓库
证据原则	优先论文原文、作者项目页、官方工程博客和GitHub仓库; 主张与推断分开
版本	v1.0 生成日期 2026-08-19

本报告不是排行榜。它是一套研究地图:

解释哪些结果可以相信、哪些只是方向性证据、哪些工程选择会改变结论。

本卷导航

01	结论先行: 年度20条核心判断
02	Agent研究对象的重新定义
03	年度技术地图与关键转折
04	八层分类体系
05	证据分级与反直觉发现
06	未来12至24个月研究议程
07	不同读者的阅读路径

1. 结论先行: 年度20条核心判断

1. Agent的最小定义已经不够

“模型加工具循环”仍是机制核心，但真实系统还包含状态、权限、执行环境、恢复、观测、验证和人机协作。年度最重要的观念变化，是把Agent当作跨层软件系统，而不是一个只会调用函数的模型。

2. Harness成为一等研究对象

同一模型在不同提示、工具、文件结构、沙箱、上下文压缩和测试协议下可能给出显著不同结果。模型排名不能直接等价于产品排名，评测必须记录完整系统配置。

3. 长时任务的瓶颈从上下文长度转向状态质量

扩大窗口只能延缓拥塞。真正决定能否跨数小时或数天推进的，是任务清单、稳定外部状态、增量产物、上下文重建、失败恢复和明确的完成判据。

4. 记忆不是数据库字段，而是未来决策的可见性合同

记忆决定未来每一步能看见什么、忽略什么、相信什么。写入错误、过度压缩、丢失来源和过期事实，比“忘记”更危险。

5. 规划从一次性计划转向分层、动态和可验证

年度论文普遍采用里程碑、递归子目标、策略图或全局-局部协同。有效计划的价值不是写得详细，而是能被环境反馈修订并与验证器连接。

6. 工具规模催生渐进披露

当工具从几十扩展到几百或上千，完整注入schema会吞噬上下文。工具检索、代码作为元工具、技能包和按需加载成为核心模式。

7. MCP解决连接，不解决信任

MCP标准化发现和调用，但不自动提供来源可信度、最小权限、语义一致性、幂等、回滚或数据边界。协议成功扩大了生态，也扩大了供应链攻击面。

8. A2A与多智能体的价值在隔离和并行，而非角色扮演

真正可测的收益来自独立上下文、并行搜索、专业工具和明确交付接口。仅给同一模型不同角色名称，常常只是增加成本和协调噪声。

9. 多智能体错误是网络传播问题

一个代理的错误摘要、错误承诺或错误状态会成为其他代理的“事实”。因此需要消息来源、置信度、责任归属、共享状态模式和失败定位。

10. 技能正在成为可审计的中间参数层

技能把程序性知识从模型权重和巨大系统提示中分离出来，可版本化、评测、回滚和按需加载。SkillOpt、MemSkill、HyperSkill等把技能进一步视为可优化对象。

11. 测试时学习正在从反思文本升级为受控演化

仅追加“经验总结”容易自我强化错误。更成熟的设计加入策略空间、失败日志、验证门、回归回滚、探索机制和跨任务留出集。

12. Agent RL的难点是稀疏奖励与信用分配

年度工作把执行轨迹转为训练转移，使用引导、形状奖励和分层信用分配。训练系统与Agent运行时解耦，是比单个算法更重要的工程趋势。

13. 评测正从最终答案转向完整轨迹

只看成功/失败无法区分工具选择、参数、顺序、恢复、验证或基础设施问题。TRAJECT-Bench、AgentRx式诊断和故障定位工作推动“最早不可恢复步骤”成为关键指标。

14. 长时可靠性不是短时准确率的线性外推

错误随步骤复合，约束随上下文稀释，代理还会过早宣布完成。短任务95%的局部成功率，不能推出百步任务可用。

15. 现实基准正在揭露能力断崖

OSWorld 2.0、WindowsWorld和跨应用办公任务表明，单应用操作改善后，跨来源推理、隐藏状态、动态信息和三应用以上协同仍会显著崩溃。

16. 基准本身已成为高风险依赖

SWE-bench系列出现污染、错误测试和基础设施噪声；网页基准还面临动态内容与评测答案泄露。领先差距小于测量误差时，排行榜没有决策意义。

17. 安全问题的核心是信息流、权限与持久状态

提示注入只是入口。真正风险是外部内容影响高权限动作、跨连接器泄露、恶意工具元数据、记忆投毒、目标漂移和无法撤销的能力。

18. 沙箱的价值是用边界换自主

逐步人工确认会造成疲劳。更好的模式是预先定义文件、网络、凭证和动作边界，在边界内允许自动执行，并对越界动作升级确认。

19. 生产差异越来越来自验证和运维

顶级模型趋近后，是否能复现状态、定位失败、重放轨迹、控制成本、处理并发、恢复中断和审计权限，决定系统是否可运营。

20. 未来竞争单位是可验证 workflow

模型仍重要，但可持续优势更可能来自领域环境、测试预言机、数据和权限连接、失败知识库、可复用技能以及组织对Agent友好的代码和文档。

一句话总结

过去一年不是Agent“突然拥有了自主意识”，而是模型、harness、执行环境和评测共同跨过了若干工程阈值。同时，长时错误、状态污染和测量失真也从边缘问题变成了主问题。

2. Agent研究对象的重新定义

本系列采用系统定义：Agent是一个在目标和约束下，持续选择信息、调用能力、改变外部状态、观察结果并依据验证信号调整行为的计算系统。这个定义有三个用途：它排除只有单轮文本生成的“Agent washing”；它把工具和环境纳入因果链；它允许对权限、恢复和责任进行工程分析。

用户与组织约束	目标、确认、预算、审计、责任
Agent harness	循环、路由、状态、上下文、恢复、可观测性
模型与策略	推理、规划、工具选择、语言/视觉动作
能力接口	API、MCP、A2A、Skills、代码、GUI
执行与状态	沙箱、文件、数据库、浏览器、任务队列
验证与治理	测试、策略、权限、监控、回滚、人工复核

核心判断: Agent是跨层系统。改变任何一层都可能改变最终分数与风险。

为什么必须跨层分析

- 模型层改进可能被工具描述不清、上下文污染或执行环境错误抵消。
- Harness优化可能只适配一个模型版本，升级后反而过度约束。
- 最终答案正确，仍可能通过违规工具、泄露数据或不可复现路径获得。
- 最终答案错误，根因可能是测试环境、依赖、网络和权限，而非推理能力。
- 因此每个Agent结论至少要标注模型、harness、工具、状态、环境、评测与成本。

Agent与workflow的边界

本系列不把两者做价值判断。Workflow预先规定控制流，优点是可预测、易审计；Agent让模型根据环境动态决定步骤，优点是处理未知路径。实际系统常是混合体：外层用确定性状态机管理预算、权限、重试和交付，内层让模型在受限动作空间中探索。越接近资金、医疗、生产变更等高风险动作，越应把关键不变量收回到确定性层。

3. 年度技术地图与关键转折

阶段	主转折	代表证据	含义
2025-08至09	现实工具和长时基准密集出现	MCPVerse、OdysseyBench、Ultra Long-Horizon	从函数调用转向数百工具、跨应用和数百步轨迹
2025-09至10	上下文工程、工具设计、技能	Anthropic context/tools/skills; MUSE; ReCAP	从塞满prompt转向按需披露、外部状态与程序性知识
2025-11至12	长时harness、演化记忆、Agent RL	长时harness; Evo-Memory; Agent Lightning	从单次代理运行转向跨会话进度和训练-运行解耦
2026-01至03	评测完整性、故障诊断、运行时分层	Agent evals; infrastructure noise; harness engineering	评测对象明确为模型加harness; 环境噪声进入主分析
2026-04至06	沙箱/计算分离、跨应用能力断崖、技能优化	Agents SDK evolution; WindowsWorld; SkillOpt	从功能演示转向运行控制、成本与可审计优化
2026-07至08	基准审计、长时职业任务、自演化结构	SWE-Bench Pro audit; OSWorld2.0; HyperSkill; Orchard	前沿问题转为数据质量、环境规模与结构化经验复用

重要提醒

所谓“年度转折”是对公开材料的研究性归纳，不意味着这些概念在该月首次出现。许多基础思想更早已先例，本系列关注的是它们在过去一年形成系统共识、工程实现或新证据的时间。

4. 八层分类体系

层	研究对象	核心问题	必须记录
1 能力模型	推理、视觉、代码、函数调用、长上下文	能否提出正确下一步	模型版本、温度、推理预算
2 Harness	循环、提示、路由、重试、压缩、子代理	能否持续推进并恢复	状态机、checkpoint、超时
3 信息与记忆	工作、事实、经历、程序、信念	未来步骤能看到什么	来源、时效、写入门、删除
4 工具与协议	API、MCP、A2A、Skills、代码、GUI	能做什么以及如何组合	schema、权限、幂等、发现
5 执行环境	沙箱、浏览器、文件、容器、队列	动作是否真实、隔离、可重放	镜像、网络、凭证、资源
6 验证	测试、规则、约束、模型裁判、人工	何时算完成或失败	oracle、覆盖率、误判、成本
7 安全治理	最小权限、监控、回滚、审计、责任	错误和攻击能造成多大影响	信息流、爆炸半径、撤销
8 组织工作流	任务拆分、评审、交付、知识资产	系统能否融入真实团队	SLA、所有权、升级路径

这八层不是按技术栈排列，而是按因果责任排列。部署故障分析应从最终错误逆向追踪每一层，直到找到最早违反不变量或失去可恢复性的步骤。

5. 证据分级与反直觉发现

级别	证据形态	示例	如何使用
A	可执行、确定性、可重放	容器化任务、真实状态检查、单位/集成测试	最适合功能正确性; 仍需控制环境噪声
B	人工审计或专家评分	盲评、独立复核、故障标注	适合开放任务; 成本高且存在主观差异
C	受控实验与消融	固定模型/工具/预算比较拓扑或记忆	可做因果推断; 外部效度有限
D	公开使用数据或运营记录	时长、adoption、故障报告	真实但选择偏差强; 不等价于生产率
E	作者/厂商演示与产品主张	案例、发布指标、客户引语	可发现方向; 需要独立复核
F	本系列推断	跨来源归纳、工程外推、趋势判断	显式标注假设和反例

六个反直觉发现

- 长上下文不一定胜过检索记忆: 当结构、说话人和时间关系被抹平时，甚至BM25会胜过复杂记忆系统。
- 树搜索不一定胜过简单分解: 如果状态评分器不稳定，搜索会放大裁判误差。
- 更多Agent不一定更好: 并行收益可能被重复检索、消息损失、合并冲突和token成本吞没。
- 更高完成率不一定更可靠: 通过脆弱测试、泄露答案或违规工具获得的成功不能代表能力。
- 更长自主时长不一定更自主: 可能只是慢、循环、等待或在错误方向上持续执行。
- 更少确认不一定更危险: 在强沙箱和清晰权限边界内，减少逐步确认可以降低确认疲劳并改善安全。

6. 未来12至24个月研究议程

方向	关键问题	建议研究设计
状态	如何定义可迁移的Agent状态schema，并区分事实、推断、承诺、计划和工具结果？	事件溯源 + provenance + 类型化状态
记忆	如何在相同token/延迟/存储预算下比较长上下文、向量、图、程序与技能记忆？	预算归一化多环境基准
规划	何时应计划、何时应直接行动、何时应重新规划？	基于不确定性和失败成本的门控策略
工具	如何对上千工具做语义发现并保证参数、权限和幂等？	分层目录 + 代码元工具 + 能力token
多Agent	如何度量并行带来的独立信息增益而非重复token？	反事实消融与共享信息账本
学习	如何防止记忆/技能自我演化过拟合近期任务或固化错误？	留出任务、回归门、回滚和探索
评测	如何让动态Web、软件和企业系统既真实又可重放？	时间冻结、代理服务、事件日志和状态快照
安全	如何撤销已分发给子代理、工具或技能的能力？	短期凭证、能力租约和持续授权检查
经济性	怎样用每个可验证成功的总成本比较系统？	token + compute + 人工复核 + 失败返工
组织	什么样的仓库、文档、接口和评审制度最“Agent友好”？	机器可读规则、测试预言机和小批量交付

优先级判断

如果只能投入三个方向，应优先做：第一，任务级确定性验证与状态重放；第二，类型化持久状态和上下文重建；第三，权限/沙箱/凭证的能力化控制。它们对模型升级具有更强的跨版本复用价值，也能直接降低误判和事故成本。

7. 不同读者的阅读路径

读者	建议顺序	理由
研究者	01 -> 03 -> 05 -> 08	先建立系统问题，再看记忆/演化、评测安全和完整索引
Agent工程师	02 -> 07 -> 05 -> 06	先看架构和上下文，再选框架，补验证与领域案例
技术负责人	01 -> 05 -> 07	先看关键判断、风险与选型，不被单一排行榜带偏
产品/业务	01 -> 06 -> 05	先识别适用任务，再评估可靠性、人工成本与风险
安全团队	05 -> 02 -> 08	先做威胁模型，再理解运行链路，最后核对原始证据

本卷结论
Agent的核心研究问题已从“模型能不能做”变成“系统在什么边界、成本和证据条件下可以稳定做”。这要求模型研究、软件工程、分布式系统、安全、HCI和组织设计共同进入同一张因果图。